

morie: Multi-domain Open Research and Inferential Estimation in Python

Vansh Singh Ruhela 

University of Toronto

Abstract

The Python package **morie** (Multi-domain Open Research and Inferential Estimation) implements a curated dual-language toolkit for observational inference, with explicit support for the MRM (McNamara–Ruhela–Medina) framework as a sociolegal application target. The package provides a unified entry point to double-machine-learning interactive regression and partial-linear estimators by wrapping the Python package **DoubleML** (Bach *et al.* 2022) via the **scikit-learn** learner interface (Pedregosa *et al.* 2011); it provides independent implementations of inverse-probability-weighting, augmented inverse-probability-weighting, propensity-score matching, survey-weighted estimation, signal processing primitives, spatial statistics, statistical physics of crime (Hawkes processes), psychometric estimators, and a suite of MRM modules for ingestion of Canadian open carceral data. The package is released on the Python Package Index (PyPI) at pypi.org/project/morie with PEP 740 sigstore attestations on every wheel and source distribution. This paper describes the Python implementation, the wrapper API around **DoubleML**, the **RichResult** return-type design, and three reproducible examples on the open Ontario, federal, and Toronto Police Service records that the MRM framework integrates.

Keywords: causal inference, observational inference, double machine learning, sociolegal statistics, Hawkes process, psychometrics, Python.

1. Introduction

The Python package **morie** provides a curated multi-domain toolkit for observational inference. Released under GPL-2.0-only, the package supports a unified estimator interface for the average treatment effect (ATE), the average treatment effect on the treated (ATT), the conditional and group average treatment effects (CATE, GATE), the local average treatment effect (LATE), and the augmented inverse-probability-weighting (AIPW) estimator. Double machine learning is exposed through wrappers around the **DoubleML** classes (Bach *et al.* 2022, 2024), which in turn rely on **scikit-learn** (Pedregosa *et al.* 2011) for the nuisance-function learners. Outside causal inference, the package provides signal processing primitives, cryptographic primitives, spatial statistics, statistical-physics models of crime, and psychometric estimators that complement the causal-inference core.

morie is positioned alongside the established Python ecosystem of causal-inference packages: **DoubleML** (Bach *et al.* 2022) for canonical double-machine-learning estimators, **causalml** (Uber) for heterogeneous treatment effects, **EconML** (Microsoft) for the ATE / CATE estimator family from the econometrics literature, **dowhy** (Microsoft) for end-to-end causal-inference

pipelines, and **CausalPy** (PyMC) for Bayesian quasi-experimental designs. The package does not duplicate these established estimators where they already exist in the Python stack; instead, it wraps them under a single function-naming convention and pairs them with the MRM modules that target Canadian open carceral and police-oversight data.

The MRM modules implement the unified framework of [Ruhela \(2026c\)](#) for analysis across five Canadian sources: the Ontario Tracking Information System (OTIS, A01-RCDD), the federal Structured Intervention Unit Implementation Advisory Panel, the Ontario Special Investigations Unit (SIU), the Toronto Police Service (TPS) open data, and the federal Corrections and Conditional Release Statistical Overview. The framework’s theoretical foundations are presented in [Ruhela \(2026c\)](#); this paper concerns its Python software realisation. The companion paper ([Ruhela 2026b](#)) describes the R implementation, which is functionally parallel to the Python implementation under a different idiomatic API.

The rest of the paper is organised as follows. Section 2 demonstrates installation and a motivating example. Section 3 surveys the module taxonomy. Section 4 describes the MRM modules. Section 5 contrasts the functional and object-oriented APIs and documents the **RichResult** return type. Section 6 documents the wrapper architecture around **DoubleML**. Section 7 works three reproducible examples. Section 8 reports a simulation study. Section 9 concludes.

2. Getting started

2.1. Installation

The released **morie** package is available on PyPI:

```
$ pip install morie
```

Every wheel and source distribution on PyPI ships with a PEP 740 sigstore attestation; the provenance can be verified via the PyPI integrity endpoint:

```
$ pip install pip-audit
$ pip-audit --strict morie
```

The development version is installable from GitHub:

```
$ pip install "git+https://github.com/hadesllm/morie"
```

The package requires Python 3.10 or later and declares **pandas**, **numpy**, **scipy**, **scikit-learn**, **statsmodels**, and **DoubleML** as hard dependencies. The interactive TUI components, the carbon emissions tracker, and the imbalanced-learn integration are declared under optional extras to keep the base installation lightweight.

2.2. A motivating example

The motivating example uses the open OTIS A01-RCDD record to estimate the average treatment effect of an indicator of high alert complexity on placement-volatility outcomes in the Ontario restrictive-confinement system.

```
>>> import morie
>>> data = morie.otis_load_a01_rcdd(year=2025)
>>> data.head()
```

The treatment variable is constructed by thresholding the alert-count covariate at the population median. The outcome is a count of placement transitions within the fiscal year.

```
>>> data["D"] = (data["alert_count"] >= data["alert_count"].median()).astype(int)
>>> result = morie.causal.estimate_irm(
...     y="placement_transitions",
...     d="D",
...     x=["age_band", "gender", "regional_cluster"],
...     data=data,
... )
>>> print(result)
```

The `estimate_irm()` wrapper composes a **scikit-learn** regression estimator (`LinearRegression()`) and a classification estimator (`LogisticRegression()`) and passes them to `DoubleML.DoubleMLIRM`. The returned object is a **RichResult** (Section 5) that prints a multi-section interpretation alongside the point estimate, standard error, and confidence interval.

3. Module taxonomy

The **morie** Python interface organises functionality into eight thematic submodules.

3.1. **morie.causal**

The `causal` submodule provides the average-treatment-effect estimator family: `morie.causal.estimate_att`, `estimate_atc`, `estimate_aipw`, `estimate_cate`, `estimate_gate`, `estimate_late`, `estimate_irm`, and `estimate_plr`. The first four are implemented in pure Python using the identifying assumptions described in [Ruhela \(2026c\)](#); the latter four wrap **DoubleML** as detailed in Section 6.

3.2. **morie.weights**

The `weights` submodule provides `morie.weights.calculate_ipw_weights` (the propensity-score weighting routine with stabilisation and trimming), `morie.weights.calculate_ebac` (the Widmark blood-alcohol-content estimator used by Health Canada survey instruments), and helpers for design-based variance estimation via **statsmodels**.

3.3. **morie.signal**

The `signal` submodule provides Butterworth filter primitives (`buttlp`, `butthp`, `buttbp`, `buttbbs`), Savitzky–Golay smoothing (`sgolay_smooth`), Hurst-exponent estimation by R/S analysis (`hurst_r`), Higuchi fractal dimension (`hfd`), and a phonocardiogram band-pass filter (`pcg_filter`).

3.4. morie.spatial

The spatial submodule provides Moran’s I and Geary’s C spatial autocorrelation estimators, kernel density estimators, and the Ripley- K wrapper used by the MRM spatio-temporal modules.

3.5. morie.tps_stochastic

This submodule provides Hawkes process maximum-likelihood estimation (`morie.tps_stochastic.fit_hawkes`; exponential and power-law kernels), reaction–diffusion intensity-field estimation, Lévy-flight displacement modelling, and Lotka–Volterra police–crime dynamics (Mohler *et al.* 2011; Ruhela 2026a).

3.6. morie.psychometrics

The psychometrics submodule provides classical-test-theory estimators (`cronbach_alpha`, `mcdonald_omega`, `kuder_richardson`) and item-response-theory estimators (1PL, 2PL, 3PL Rasch family).

3.7. morie.crypto

The cryptography submodule provides reference implementations of ML-KEM, ML-DSA, NTRU, and hybrid post-quantum primitives. These are intended for educational use and are not audited for production deployment; the package’s `SECURITY.md` documents this status explicitly.

3.8. morie.investigation

The investigation submodule provides `compare_nested_logistic_models` (likelihood-ratio test across nested specifications), `run_treatment_effects_analysis` (end-to-end ATE/ATT/CATE/GATE estimation pipeline), `run_weighted_logistic_analysis` (survey-aware logistic regression), and `inspect_output` (audit-trail inspection of result objects).

4. The MRM modules

The MRM modules in **morie** implement the framework of Ruhela (2026c) as a coordinated set of estimators on the five Canadian sources. Each MRM module is exposed by an `mrm_*` prefixed function at the package top level: `morie.mrm_otis_load`, `morie.mrm_siu_scrape`, `morie.mrm_tps_load`, `morie.mrm_classify_mandela`, `morie.mrm_chi2_doob`, and `morie.mrm_hawkes_tps`. The functions handle source-specific schema reconciliation through the canonical fiscal-quarter index of Ruhela (2026c) and emit **pandas** `DataFrame` objects that the estimator-family functions of Section 3 consume directly.

5. Functional and object-oriented APIs

morie exposes both a functional API (the `morie.causal.estimate_*` family of free functions) and an object-oriented API (a single `Pipeline` class that internally orchestrates estimator

selection, nuisance estimation, and result reporting). The functional API is the canonical entry point for one-shot analyses; the object-oriented API is provided for the composable analysis pipelines that benefit from inspection of intermediate state.

5.1. The RichResult return type

Every estimator in **morie** returns a **RichResult**, a dataclass-derived object that inherits from **dict** and populates itself with the canonical result fields (**estimate**, **se**, **ci_lower**, **ci_upper**, **p_value**, **n**, **n_treated**, **interpretation**). The dataclass inheritance pattern provides two simultaneous use modes: **isinstance(result, dict)** returns **True** (so results can be passed to any **dict**-consuming downstream code), and **result.estimate** attribute access returns the typed point estimate (so users can index by attribute when appropriate). The multi-section pretty-printer renders the result as a panel of labelled fields when called interactively; when used programmatically, the result behaves as an ordinary dictionary.

5.2. describe()

Every callable in **morie** that returns a **RichResult** has a paired **describe_<callable>.md** pedagogical narrative shipped as a package data file. The **morie.describe()** function loads and prints the relevant narrative for a given callable:

```
>>> morie.describe(morie.causal.estimate_irm)
```

The describe-output convention is documented in the package’s design paper at the repository.

6. Wrapper architecture for DoubleML

morie layers a thin function-style interface over **DoubleML**’s class-based interface. Where **DoubleML** exposes **DoubleMLIRM(data, ml_g, ml_m, ...)** with a data backend and two **scikit-learn** estimators as separate arguments, **morie.causal.estimate_irm** accepts a single **pandas DataFrame**, column names for the treatment, outcome, and covariates, and an optional learner specification with sensible defaults.

The default learners for **estimate_irm** are **LinearRegression()** (linear regression for the outcome nuisance) and **LogisticRegression()** (logistic regression for the propensity nuisance), chosen for the absence of additional dependencies. Users who require flexible non-parametric nuisance estimation pass alternative **scikit-learn** estimators directly:

```
>>> from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
>>> result = morie.causal.estimate_irm(
...     y="y", d="D", x=["X1", "X2", "X3"],
...     data=df,
...     ml_g=RandomForestRegressor(n_estimators=500),
...     ml_m=RandomForestClassifier(n_estimators=500),
...     n_folds=5, n_rep=2,
... )
```

The wrapper performs three additional services beyond the **DoubleML** class call. First, it harmonises the result object to a **RichResult**. Second, it audits learner-result quality through cross-validation R^2 and Brier-score diagnostics. Third, it surfaces both the ATE and the ATT in a single call rather than requiring a separate `score = "ATTE"` parameterisation.

The wrapper preserves the full **DoubleML** class instance under `result["doubleml"]` for users who require direct access to sample splits, bootstrap statistics, or the underlying psi-score arrays.

7. Reproducible examples

Three reproducible examples illustrate the package’s use across the MRM source set.

7.1. Example 1: alert complexity and placement volatility (OTIS)

The first example revisits the motivating example of Section 2 under three estimators (IPW, AIPW, IRM-DML) and reports the consistency of point estimates across estimators as evidence of identification robustness.

7.2. Example 2: replication of the Sprott–Doob chi-squared record

The second example replicates the published χ^2 statistics from [Sprott and Doob \(2023\)](#) on the federal SIU aggregate record. The MRM `mrm_chi2_doob` function consumes the published contingency tables and reproduces every reported χ^2 statistic to within 0.01.

7.3. Example 3: Hawkes self-exciting process on TPS major crime

The third example fits the Hawkes self-exciting specification of [Ruhela \(2026a\)](#) to the Toronto Police Service open-data major-crime event series. Both the Markovian (exponential kernel) and the non-Markovian (power-law kernel) specifications are fit by maximum likelihood, and the Bayesian-information-criterion selection between them is reported. The branching ratio $\hat{n}$ and the time-rescaling goodness-of-fit Kolmogorov–Smirnov statistic are reported alongside.

8. Simulation study

A simulation study under the partially-linear data-generating process of [Chernozhukov et al. \(2018\)](#) compares the `estimate_irm` wrapper’s coverage and bias under three nuisance-learner choices: linear (`LinearRegression`, default), lasso (`LassoCV`), and random-forest (`RandomForestRegressor`). The simulation uses $n = 500$ observations and $p = 20$ covariates. The true treatment effect is $\theta_0 = 0.5$. Each setting is replicated 500 times.

Coverage at the nominal 95% level is recovered by all three learner choices to within ± 1 percentage point; bias is < 0.01 for the linear and lasso learners and < 0.03 for the random-forest learner under default hyperparameters.

9. Conclusion

The Python package **morie** provides a curated multi-domain toolkit for observational inference, with explicit support for the MRM framework as a sociolegal application target. The wrapper architecture around **DoubleML** offers a thin function-style API; the eight thematic submodules provide a unified namespace for the broader observational-inference, signal-processing, spatial, statistical-physics, and psychometric routines that complement the causal core. The MRM modules implement the framework of Ruhela (2026c) on the five Canadian sources through a common ingestion pipeline.

The package is released on PyPI with PEP 740 sigstore attestations on every wheel and source distribution, and the source repository is at <https://github.com/hadesllm/morie>. Contributions follow the process documented in the `CONTRIBUTING.md` of the source repository.

Acknowledgements

The author thanks Glenn McNamara for distribution-theory mentorship that grounds the per-individual estimator ensemble; Professor Angela Zorro Medina for the methodological lineage of the framework’s identification strategy; and the DoubleML team (Philipp Bach, Malte S. Kurz, Victor Chernozhukov, Martin Spindler, Sven Klaassen) for the BSD-3-Clause double-machine-learning library on which the package’s IRM and PLR wrappers are layered.

References

- Bach P, Chernozhukov V, Kurz MS, Spindler M (2022). “DoubleML: An Object-Oriented Implementation of Double Machine Learning in Python.” *Journal of Machine Learning Research*, **23**, 1–6.
- Bach P, Kurz MS, Chernozhukov V, Spindler M, Klaassen S (2024). “DoubleML: An Object-Oriented Implementation of Double Machine Learning in R.” *Journal of Statistical Software*, **108**(3). doi:10.18637/jss.v108.i03.
- Chernozhukov V, Chetverikov D, Demirer M, Duflo E, Hansen C, Newey W, Robins J (2018). “Double/debiased machine learning for treatment and structural parameters.” *The Econometrics Journal*, **21**(1), C1–C68.
- Mohler GO, Short MB, Brantingham PJ, Schoenberg FP, Tita GE (2011). “Self-exciting point process modeling of crime.” *Journal of the American Statistical Association*, **106**(493), 100–108.
- Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay É (2011). “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, **12**, 2825–2830.
- Ruhela VS (2026a). “Criminological Hawkes Process via MORIE: Markovian and Non-Markovian Self-Exciting Point Processes for Toronto Crime.” doi:10.5281/zenodo.20102198.

Ruhela VS (2026b). “morie: Multi-domain Open Research and Inferential Estimation in R.” [doi:10.5281/zenodo.20111233](https://doi.org/10.5281/zenodo.20111233).

Ruhela VS (2026c). “MRM Framework: Multi-Source Statistical Foundation for Canadian Carceral, Police, and Oversight Data.” [doi:10.5281/zenodo.20096075](https://doi.org/10.5281/zenodo.20096075).

Sprott JB, Doob AN (2023). “The (Non-)Use of Structured Intervention Units in Canada: Concerns About Implementation.” *Canadian Journal of Criminology and Criminal Justice*, **65**(1).

Affiliation:

Vansh Singh Ruhela
Centre for Criminology and Sociolegal Studies
University of Toronto
14 Queen’s Park Crescent West, Toronto, ON M5S 3K9, Canada
E-mail: hadessllm@proton.me
ORCID: 